



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS DE ARARANGUÁ

Ana Luiza Fedyszn Kruger, Eloá Brizeis Costa Santana e Giulia Guizzo Baladão Santos

Trabalho 1 - Recursividade

Araranguá

2026

1 INTRODUÇÃO

A recursividade é um conceito fundamental em Tecnologias da Informação e Comunicação, no qual uma função resolve um problema chamando a si mesma com uma versão menor ou mais simples do mesmo problema. Para que a recursão funcione corretamente, toda função recursiva deve possuir dois elementos essenciais: o caso base, que interrompe as chamadas e evita recursão infinita, e o passo recursivo, que reduz o problema a cada chamada até atingir o caso base.

Este relatório documenta a implementação de três algoritmos recursivos em Python, desenvolvidos como parte do Trabalho 1 da disciplina de Fundamentos Matemáticos para Computação: Torres de Hanói (Exercício 3), Inversão de String (Exercício 4) e Conversão Decimal para Binário (Exercício 5). Para cada algoritmo, são apresentados o código-fonte, a identificação do caso base, a explicação do passo recursivo e o detalhamento da sequência de chamadas e retornos da função.

2 TORRES DE HANÓI

2.1 Descrição do Problema

O problema das Torres de Hanói consiste em mover N discos empilhados do menor ao maior de um pino de origem para um pino de destino, utilizando um pino auxiliar. A regra fundamental é que um disco maior nunca pode ser colocado sobre um menor. O número mínimo de movimentos necessários para N discos é $2^n - 1$.

2.2 Implementação

```
def hanoi(n, origem, destino, auxiliar):  
    if n == 1: # caso base  
        print(f"Mover disco 1 de {origem} para {destino}")  
        return  
    hanoi(n - 1, origem, auxiliar, destino)  
    print(f"Mover disco {n} de {origem} para {destino}")  
    hanoi(n - 1, auxiliar, destino, origem)
```

2.3 Caso Base

O caso base ocorre quando $n = 1$. Nessa situação, há apenas um disco para mover, e ele é transferido diretamente do pino de origem para o pino de destino, sem necessidade de recursão. Sem esse caso base, a função chamaria a si mesma indefinidamente.

2.4 Passo Recursivo

A lógica recursiva segue três etapas obrigatórias:

- Mover os $N-1$ discos superiores do pino de origem para o pino auxiliar, usando o destino como intermediário.
- Mover o disco N (o maior restante) diretamente do pino de origem para o pino de destino.
- Mover os $N-1$ discos do pino auxiliar para o pino de destino, usando a origem como intermediário.

2.5 Rastreio Manual ($N = 3$)

hanoi(3, A, C, B)

hanoi(2, A, B, C) → move $N-1$ discos de A para B

hanoi(1, A, C, B) → Mover disco 1 de A para C

Mover disco 2 de A para B

hanoi(1, C, B, A) → Mover disco 1 de C para B

Mover disco 3 de A para C

hanoi(2, B, C, A) → move $N-1$ discos de B para C

hanoi(1, B, A, C) → Mover disco 1 de B para A

Mover disco 2 de B para C

hanoi(1, A, C, B) → Mover disco 1 de A para C

Total: 7 movimentos ($2^3 - 1 = 7$).

3 INVERSÃO DE STRING

3.1 Descrição do Problema

O problema consiste em receber uma string e retornar seus caracteres em ordem invertida. Por exemplo, a entrada "ufsc" deve produzir a saída "csfu". A abordagem recursiva resolve o problema isolando um caractere por vez e delegando a inversão do restante para a próxima chamada.

3.2 Implementação

```
def inverter_string(s):
    if len(s) <= 1: # caso base
        return s
    return s[-1] + inverter_string(s[:-1])
```

3.3 Caso Base

O caso base ocorre quando a string possui zero ou um caractere ($\text{len}(s) \leq 1$). Uma string vazia ou de um único caractere já é, por definição, o seu próprio inverso. Nesses casos, a função retorna s sem realizar nenhuma chamada recursiva.

3.4 Passo Recursivo

Para inverter uma string de tamanho N , a função isola o último caractere ($s[-1]$) e o concatena com o resultado da inversão do restante da string sem o último caractere ($s[:-1]$), que possui tamanho $N-1$. A relação de recorrência pode ser expressa como:

$$\text{inverter_string}(s) = s[-1] + \text{inverter_string}(s[:-1])$$

A cada chamada, o problema diminui em exatamente 1 caractere, garantindo que a recursão sempre alcance o caso base.

3.5 Rastreio Manual ($s = \text{"ufsc"}$)

Empilhamento (chamadas):

```
inverter_string("ufsc") → "c" + inverter_string("ufs")
inverter_string("ufs") → "s" + inverter_string("uf")
inverter_string("uf") → "f" + inverter_string("u")
```

```
inverter_string("u") → "u" [caso base]
```

Desempilhamento (retornos):

```
inverter_string("uf") ← "f" + "u" = "fu"
inverter_string("ufs") ← "s" + "fu" = "sfu"
inverter_string("ufsc") ← "c" + "sfu" = "csfu"
Resultado final: "csfu".
```

4 CONVERSÃO DECIMAL PARA BINÁRIO

4.1 Descrição do Problema

O problema consiste em converter um número inteiro não negativo da base decimal para a representação em base binária (base 2). Por exemplo, o número decimal 13 deve ser convertido para "1101". A conversão é feita por divisões sucessivas por 2, capturando os restos, que formam os dígitos binários de trás para frente.

4.2 Implementação

```
def decimal_para_binario(n):
    if n <= 1: # caso base
        return str(n)
    return decimal_para_binario(n // 2) + str(n % 2)
```

4.3 Caso Base

O caso base ocorre quando $n \leq 1$, ou seja, quando n vale 0 ou 1. Nesses casos, o número já corresponde diretamente ao seu valor binário ("0" ou "1"), e a função retorna `str(n)` sem realizar novas chamadas. Sem esse caso base, a divisão por 2 nunca atingiria o fim e a recursão seria infinita.

4.4 Passo Recursivo

Para converter n , a função divide n por 2 inteiramente ($n // 2$) e captura o resto ($n \% 2$). O resultado da conversão recursiva de $n // 2$ é colocado antes do resto atual, garantindo a ordem correta dos bits. A relação de recorrência é:

```
decimal_para_binario(n) = decimal_para_binario(n // 2) + str(n % 2)
```

A cada chamada, n é reduzido à metade (divisão inteira), garantindo que a recursão sempre converge para o caso base. O resto (0 ou 1) é convertido para string com `str()` para que possa ser concatenado com o resultado da chamada anterior.

4.5 Rastreio Manual ($n = 13$)

Empilhamento (chamadas):

```
decimal_para_binario(13) → 13//2=6, 13%2=1
decimal_para_binario(6)  → 6//2=3, 6%2=0
decimal_para_binario(3)  → 3//2=1, 3%2=1
decimal_para_binario(1)  → "1" [caso base]
```

Desempilhamento (retornos):

```
decimal_para_binario(3) ← "1" + "1" = "11"
decimal_para_binario(6) ← "11" + "0" = "110"
decimal_para_binario(13) ← "110" + "1" = "1101"
```

Resultado final: "1101" (13 em binário).

5 CONCLUSÃO

Os três algoritmos implementados demonstram como a recursividade pode expressar soluções elegantes e concisas para problemas que, de forma iterativa, exigiriam estruturas de controle mais complexas. Em todos os casos, a chave para o correto funcionamento foi a definição precisa do caso base, que garante a terminação da recursão, e do passo recursivo, que reduz o problema a uma instância menor a cada chamada.

No algoritmo de Torres de Hanói, a recursão se manifesta nos três passos clássicos, movendo sempre $N-1$ discos para liberar o maior. Na Inversão de String, cada chamada isola o último caractere e delega o restante, construindo o resultado durante o retorno das chamadas. Na Conversão Decimal para Binário, as divisões sucessivas empilham os restos na ordem inversa, e o desempilhamento monta os bits na sequência correta.

A análise dos rastreios manuais evidencia o mecanismo de pilha de chamadas: cada chamada recursiva é empilhada aguardando o resultado da próxima, e o valor final é construído progressivamente durante a fase de desempilhamento (retorno). Esse entendimento é essencial para aplicar a recursividade de forma segura e eficiente em problemas computacionais.

REFERÊNCIAS

BARRETO, J. M. **Fundamentos Matemáticos para Computação**. Apostila da disciplina DEC7121. Universidade Federal de Santa Catarina, Araranguá, 2026.

CORMEN, T. H. et al. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

GERSTING, J. L. **Mathematical Structures for Computer Science**. 3. ed. New York: Computer Science Press, 1993.

Python Software Foundation. **Python 3 Documentation**. Disponível em: <https://docs.python.org/3/>. Acesso em: jun. 2026.
